

```
#PART 1 – Install everything in Colab
!pip install -q \
    openai \
    langchain-openai \
    langgraph
```

```
#PART 2 – Imports
import os
import json
import time
import random

from pprint import pprint
from typing import (
    TypedDict,
    List,
    Dict,
    Any,
    Optional
)

#LLM
from langchain_openai import ChatOpenAI

#langGraph
from langgraph.graph import (
    StateGraph,
    END
)
```

```
# for MCP
!pip install -q -U "mcp[cli]>=2"
```

```
import mcp
from importlib.metadata import version

print("MCP version:", version("mcp"))
```

MCP version: 2.1.1

```
#Create the MCP 2.x Server
from mcp.server.mcpserver import MCPServer

mcp_server = MCPServer(
    "Incident Investigation Server",
    description="MCP server exposing enterprise incident-management capabilities."
)

print("MCP Server created")
```

MCP Server created

```
import getpass

os.environ["OPENAI_API_KEY"] = getpass.getpass(
    "Enter OpenAI API key: "
)
```

Enter OpenAI API key:

```
# Create Mock Enterprise Data
INCIDENTS = {
    "INC-2045": {
        "incident_id": "INC-2045",
        "application": "Checkout-Service",
        "server": "APP-PROD-12",
        "severity": "P1",
        "description": "Checkout response time increased from 1.2 sec to 8.7 sec"
    }
}
```

```

}

SERVER_METRICS = {
    "APP-PROD-12": {
        "cpu": 96,
        "memory": 71,
        "db_connections": 100,
        "error_rate": 18,
        "status": "DEGRADED"
    }
}

HISTORICAL_INCIDENTS = [
    {
        "incident_id": "INC-1781",
        "symptom": "High latency and DB connections at 100%",
        "root_cause": "Database connection pool exhaustion",
        "resolution": "Restart application connection pool and increase pool size"
    },
    {
        "incident_id": "INC-1654",
        "symptom": "High CPU and slow checkout",
        "root_cause": "Expensive SQL query",
        "resolution": "Optimize SQL query and add missing database index"
    }
]

RUNBOOKS = {

    "connection_pool": """
Connection Pool Runbook

1. Check active database connections.
2. Verify connection pool utilization.
3. Check database availability.
4. Review application logs.
5. Restart application connection pool if approved.
6. Monitor latency after remediation.
""",

    "high_cpu": """
High CPU Runbook

1. Identify high CPU process.
2. Check recent deployments.
3. Inspect expensive database queries.
4. Check application thread utilization.
5. Review CPU trend.
6. Escalate if CPU remains above threshold.
""",
}

```

```

#Create the Original Python Functions
def get_incident(incident_id: str) -> dict:
    """
    Retrieve an incident by incident ID.
    """

    return INCIDENTS.get(
        incident_id,
        {
            "error": "Incident not found",
            "incident_id": incident_id
        }
    )

def get_server_metrics(server: str) -> dict:
    """
    Retrieve operational metrics for a server.
    """

```

```

    return SERVER_METRICS.get(
        server,
        {
            "error": "Server not found",
            "server": server
        }
    )

def search_previous_incidents(keyword: str) -> list:
    """
    Search previous incidents using a keyword.
    """

    keyword = keyword.lower()

    results = []

    for incident in HISTORICAL_INCIDENTS:

        incident_text = str(incident).lower()

        if keyword in incident_text:
            results.append(incident)

    return results

def search_runbook(topic: str) -> str:
    """
    Search operational runbooks.
    """

    topic = topic.lower()

    if "connection" in topic or "database" in topic:
        return RUNBOOKS["connection_pool"]

    if "cpu" in topic:
        return RUNBOOKS["high_cpu"]

    return "No matching runbook found."

```

#Test them before introducing MCP:

```

print(get_incident("INC-2045"))
print()

```

```

print(get_server_metrics("APP-PROD-12"))
print()

```

```

print(search_previous_incidents("connection"))
print()

```

```

print(search_runbook("connection pool"))

```

```

{'incident_id': 'INC-2045', 'application': 'Checkout-Service', 'server': 'APP-PROD-12', 'severity': 'P1', 'description': 'Checko

```

```

{'cpu': 96, 'memory': 71, 'db_connections': 100, 'error_rate': 18, 'status': 'DEGRADED'}

```

```

[{'incident_id': 'INC-1781', 'symptom': 'High latency and DB connections at 100%', 'root_cause': 'Database connection pool exhau

```

Connection Pool Runbook

1. Check active database connections.
2. Verify connection pool utilization.
3. Check database availability.
4. Review application logs.
5. Restart application connection pool if approved.
6. Monitor latency after remediation.

#Expose Incident Lookup as an MCP Tool

```

@mcp_server.tool()
def mcp_get_incident(incident_id: str) -> dict:
    """
    Retrieve information about an IT incident.

```

```

Args:
    incident_id: Incident identifier such as INC-2045.
"""

return get_incident(incident_id)

```

get_incident()

↓

@mcp_server.tool()

↓

Discoverable MCP capability

```

#Expose Server Metrics as an MCP Tool
@mcp_server.tool()
def mcp_get_server_metrics(server: str) -> dict:
    """
    Retrieve server operational metrics.

    Args:
        server: Server name such as APP-PROD-12.
    """

    return get_server_metrics(server)

```

```

#Expose Historical Incident Search
@mcp_server.tool()
def mcp_search_incidents(keyword: str) -> list:
    """
    Search historical incidents.

    Args:
        keyword: Search term such as connection, CPU, latency, database.
    """

    return search_previous_incidents(keyword)

```

```

#Expose Runbook Search
@mcp_server.tool()
def mcp_get_runbook(topic: str) -> str:
    """
    Retrieve an operational runbook.

    Args:
        topic: Operational topic such as connection pool or high CPU.
    """

    return search_runbook(topic)

```

```

#Discover Available MCP Tools
tools = await mcp_server.list_tools()

print("AVAILABLE MCP TOOLS")
print("=" * 50)

for number, tool in enumerate(tools, start=1):

    print(f"{number}. {tool.name}")

    if tool.description:
        print("    ", tool.description)

    print()

```

```

AVAILABLE MCP TOOLS
=====
1. mcp_search_incidents

```

Search historical incidents.

Args:

keyword: Search term such as connection, CPU, latency, database.

2. mcp_get_server_metrics

Retrieve server operational metrics.

Args:

server: Server name such as APP-PROD-12.

3. mcp_get_incident

Retrieve information about an IT incident.

Args:

incident_id: Incident identifier such as INC-2045.

4. mcp_get_runbook

Retrieve an operational runbook.

Args:

topic: Operational topic such as connection pool or high CPU.

```
#Inspect the Tool Schemas
#MCP derives input schemas from your Python type hints.
from pprint import pprint

for tool in tools:

    print("\nTOOL:", tool.name)

    print("Description:")
    print(tool.description)

    print("Input Schema:")

    pprint(tool.input_schema)
```

TOOL: mcp_search_incidents

Description:

Search historical incidents.

Args:

keyword: Search term such as connection, CPU, latency, database.

Input Schema:

```
{'properties': {'keyword': {'title': 'Keyword', 'type': 'string'}},
 'required': ['keyword'],
 'title': 'mcp_search_incidentsArguments',
 'type': 'object'}
```

TOOL: mcp_get_server_metrics

Description:

Retrieve server operational metrics.

Args:

server: Server name such as APP-PROD-12.

Input Schema:

```
{'properties': {'server': {'title': 'Server', 'type': 'string'}},
 'required': ['server'],
 'title': 'mcp_get_server_metricsArguments',
 'type': 'object'}
```

TOOL: mcp_get_incident

Description:

Retrieve information about an IT incident.

Args:

```

    incident_id: Incident identifier such as INC-2045.

Input Schema:
{'properties': {'incident_id': {'title': 'Incident Id', 'type': 'string'}},
 'required': ['incident_id'],
 'title': 'mcp_get_incidentArguments',
 'type': 'object'}

TOOL: mcp_get_runbook
Description:

Retrieve an operational runbook.

Args:
    topic: Operational topic such as connection pool or high CPU.

Input Schema:
{'properties': {'topic': {'title': 'Topic', 'type': 'string'}},
 'required': ['topic'],
 'title': 'mcp_get_runbookArguments',
 'type': 'object'}

```

```

#Call an MCP Tool
result = await mcp_server.call_tool(
    "mcp_get_incident",
    {
        "incident_id": "INC-2045"
    }
)

print(result)

```

```
meta=None content=[TextContent(type='text', text='{
  "incident_id": "INC-2045",
  "application": "Checkout-Service",
  "serv
```

```

#inspect it
from pprint import pprint

pprint(result)

```

```
CallToolResult(meta=None, content=[TextContent(type='text', text='{
  "incident_id": "INC-2045",
  "application": "Checkout-Se
```

```

#Call the Server Metrics Tool
metrics_result = await mcp_server.call_tool(
    "mcp_get_server_metrics",
    {
        "server": "APP-PROD-12"
    }
)

pprint(metrics_result)

```

```
CallToolResult(meta=None, content=[TextContent(type='text', text='{
  "cpu": 96,
  "memory": 71,
  "db_connections": 100,

```

```

#Search Historical Incidents
history_result = await mcp_server.call_tool(
    "mcp_search_incidents",
    {
        "keyword": "connection"
    }
)

pprint(history_result)

```

```
CallToolResult(meta=None, content=[TextContent(type='text', text='{
  "incident_id": "INC-1781",
  "symptom": "High latency an
```

Build a Simple MCP-Based Investigation

Now use the exposed capabilities together.

```

async def investigate_incident_with_mcp(incident_id: str):

    print("=" * 60)
    print("MCP INCIDENT INVESTIGATION")

```

```

print("=" * 60)

# -----
# Step 1 – Get incident
# -----

print("\n[1] Retrieving incident")

incident_result = await mcp_server.call_tool(
    "mcp_get_incident",
    {
        "incident_id": incident_id
    }
)

print(incident_result)

# For our demo we already know the server from mock data.
incident = get_incident(incident_id)

if "error" in incident:
    print("Incident not found")
    return

server = incident["server"]

# -----
# Step 2 – Get server metrics
# -----

print("\n[2] Collecting metrics")

metrics_result = await mcp_server.call_tool(
    "mcp_get_server_metrics",
    {
        "server": server
    }
)

print(metrics_result)

metrics = get_server_metrics(server)

# -----
# Step 3 – Reason about metrics
# -----

print("\n[3] Evaluating current state")

if metrics["db_connections"] >= 95:

    investigation_topic = "connection pool"

    print(
        "Database connections are critically high:",
        metrics["db_connections"],
        "%"
    )

elif metrics["cpu"] >= 90:

    investigation_topic = "high cpu"

    print(
        "CPU is critically high:",
        metrics["cpu"],
        "%"
    )

else:

    investigation_topic = "general"

```

```

        print("No dominant signal detected")

# -----
# Step 4 – Search history
# -----

print("\n[4] Searching historical incidents")

history_result = await mcp_server.call_tool(
    "mcp_search_incidents",
    {
        "keyword": investigation_topic
    }
)

print(history_result)

# -----
# Step 5 – Retrieve runbook
# -----

print("\n[5] Retrieving runbook")

runbook_result = await mcp_server.call_tool(
    "mcp_get_runbook",
    {
        "topic": investigation_topic
    }
)

print(runbook_result)

print("\nInvestigation completed.")

```

```

#Run it:
await investigate_incident_with_mcp("INC-2045")

```

```

=====
MCP INCIDENT INVESTIGATION
=====

[1] Retrieving incident
meta=None content=[TextContent(type='text', text='{
  "incident_id": "INC-2045",
  "application": "Checkout-Service",
  "serv

[2] Collecting metrics
meta=None content=[TextContent(type='text', text='{
  "cpu": 96,
  "memory": 71,
  "db_connections": 100,
  "error_rate": 18

[3] Evaluating current state
Database connections are critically high: 100 %

[4] Searching historical incidents
meta=None content=[TextContent(type='text', text='{
  "incident_id": "INC-1781",
  "symptom": "High latency and DB connections

[5] Retrieving runbook
meta=None content=[TextContent(type='text', text='\nConnection Pool Runbook\n\n1. Check active database connections.\n2. Verify

Investigation completed.

```

```

#Make the Demo Output More Human-Friendly
async def run_mcp_capstone(incident_id: str):

    print("\n" + "=" * 65)
    print("          AGENTIC INCIDENT INVESTIGATION USING MCP")
    print("=" * 65)

    # INCIDENT
    incident = get_incident(incident_id)

    print("\n[SUPERVISOR]")
    print("Investigation started:", incident_id)

    print("\n[INCIDENT TOOL]")

```



```

print("Application :", incident["application"])
print("Server      :", incident["server"])
print("Severity    :", incident["severity"])
print("Problem     :", incident["description"])

# Call through MCP
await mcp_server.call_tool(
    "mcp_get_incident",
    {
        "incident_id": incident_id
    }
)

# METRICS
await mcp_server.call_tool(
    "mcp_get_server_metrics",
    {
        "server": incident["server"]
    }
)

metrics = get_server_metrics(
    incident["server"]
)

print("\n[MONITORING AGENT]")
print("CPU           :", metrics["cpu"], "%")
print("Memory        :", metrics["memory"], "%")
print("DB Connections :", metrics["db_connections"], "%")
print("Error Rate     :", metrics["error_rate"], "%")
print("Status        :", metrics["status"])

# CONDITIONAL REASONING
if metrics["db_connections"] >= 95:

    topic = "connection pool"

    hypothesis = (
        "Database connection pool exhaustion"
    )

elif metrics["cpu"] >= 90:

    topic = "high cpu"

    hypothesis = (
        "CPU saturation"
    )

else:

    topic = "general"

    hypothesis = (
        "Insufficient evidence"
    )

print("\n[SUPERVISOR]")
print("Selected investigation path:", topic)

# MEMORY
await mcp_server.call_tool(
    "mcp_search_incidents",
    {
        "keyword": "connection"
    }
)

historical = search_previous_incidents(
    "connection"
)

```

```

print("\n[KNOWLEDGE AGENT]")

if historical:

    previous = historical[0]

    print(
        "Similar Incident :",
        previous["incident_id"]
    )

    print(
        "Previous Cause  :",
        previous["root_cause"]
    )

    print(
        "Previous Fix      :",
        previous["resolution"]
    )

else:

    print("No similar incidents found")

# RUNBOOK
await mcp_server.call_tool(
    "mcp_get_runbook",
    {
        "topic": topic
    }
)

runbook = search_runbook(topic)

print("\n[RUNBOOK]")
print(runbook)

# DIAGNOSIS
confidence = 0.90

recommendation = (
    "Verify connection pool utilization. "
    "If confirmed, restart application "
    "connection pool and monitor latency."
)

print("\n[DIAGNOSIS AGENT]")

print(
    "Likely Root Cause:",
    hypothesis
)

print(
    "Confidence:",
    confidence
)

print(
    "Recommendation:",
    recommendation
)

# RISK CHECK
if "restart" in recommendation.lower():

    risk_level = "HIGH"
    approval_status = "REQUIRED"

```

```

else:

    risk_level = "LOW"
    approval_status = "NOT_REQUIRED"

print("\n[RISK CHECK]")

print(
    "Risk Level:",
    risk_level
)

print(
    "Human Approval:",
    approval_status
)

# FINAL STATE
final_state = {

    "incident_id": incident_id,

    "incident": incident,

    "metrics": metrics,

    "hypothesis": hypothesis,

    "historical_incidents":
        historical,

    "runbook": runbook,

    "confidence": confidence,

    "recommendation":
        recommendation,

    "risk_level":
        risk_level,

    "approval_status":
        approval_status,

    "status":
        (
            "WAITING_FOR_APPROVAL"
            if approval_status == "REQUIRED"
            else "READY_FOR_ACTION"
        )
}

print("\n" + "=" * 65)
print("FINAL STATE")
print("=" * 65)

return final_state

```

```

#Run
final_state = await run_mcp_capstone(
    "INC-2045"
)

from pprint import pprint

pprint(final_state)

```

```

4. Review application logs.
5. Restart application connection pool if approved.
6. Monitor latency after remediation.

[DIAGNOSIS AGENT]
Likely Root Cause: Database connection pool exhaustion
Confidence: 0.9
Recommendation: Verify connection pool utilization. If confirmed, restart application connection pool and monitor latency.

[RISK CHECK]
Risk Level: HIGH
Human Approval: REQUIRED

=====
FINAL STATE
=====
{'approval_status': 'REQUIRED',
 'confidence': 0.9,
 'historical_incidents': [{'incident_id': 'INC-1781',
                           'resolution': 'Restart application connection pool '
                           'and increase pool size',
                           'root_cause': 'Database connection pool exhaustion',
                           'symptom': 'High latency and DB connections at '
                           '100%'}],
 'hypothesis': 'Database connection pool exhaustion',
 'incident': {'application': 'Checkout-Service',
              'description': 'Checkout response time increased from 1.2 sec to '
              '8.7 sec',
              'incident_id': 'INC-2045',
              'server': 'APP-PROD-12',
              'severity': 'P1'},
 'incident_id': 'INC-2045',
 'metrics': {'cpu': 96,
             'db_connections': 100,
             'error_rate': 18,
             'memory': 71,
             'status': 'DEGRADED'},
 'recommendation': 'Verify connection pool utilization. If confirmed, restart '
 'application connection pool and monitor latency.',
 'risk_level': 'HIGH',
 'runbook': '\n'
            'Connection Pool Runbook\n'
            '\n'
            '1. Check active database connections.\n'
            '2. Verify connection pool utilization.\n'
            '3. Check database availability.\n'
            '4. Review application logs.\n'
            '5. Restart application connection pool if approved.\n'
            '6. Monitor latency after remediation.\n',
 'status': 'WAITING_FOR_APPROVAL'}

```

The business functions themselves did not need to change. MCP standardized how those capabilities are described, discovered, and invoked.

Start coding or [generate](#) with AI.